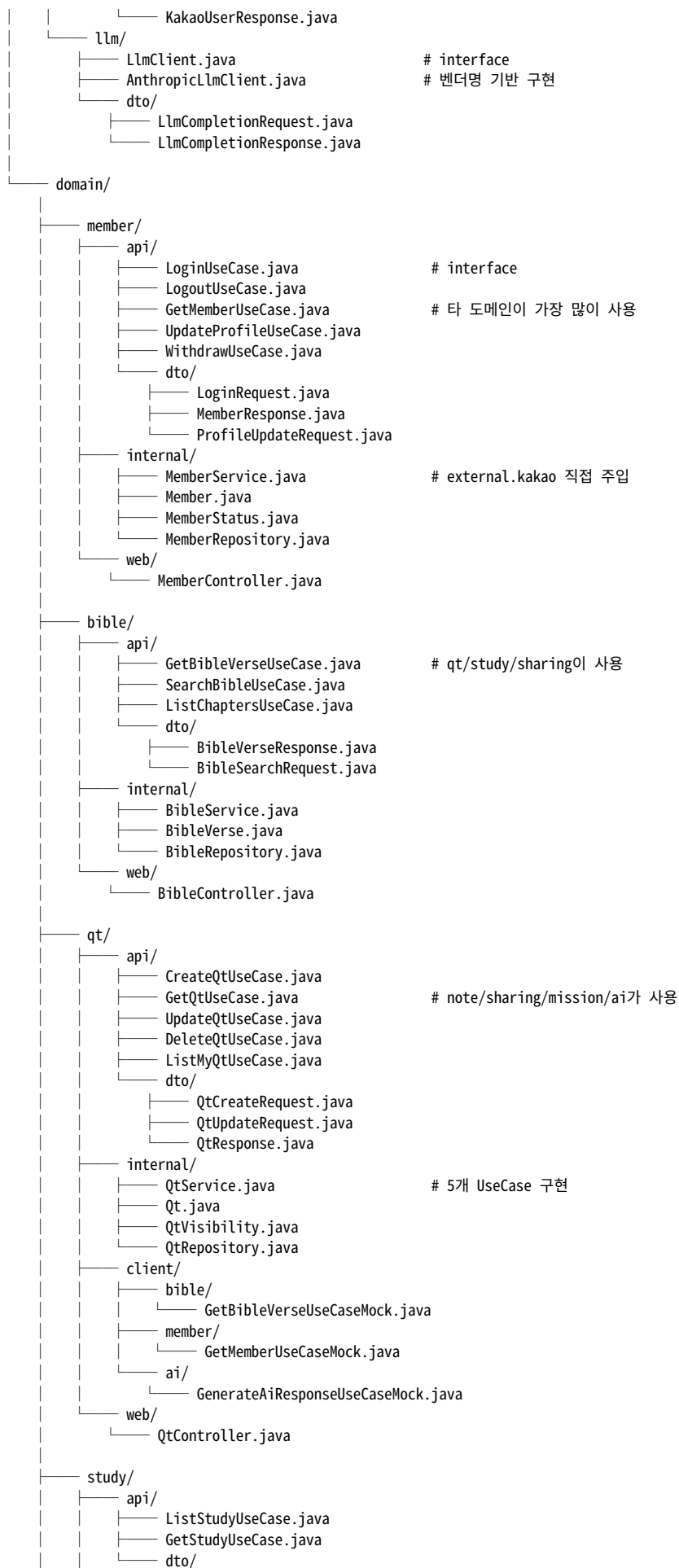


QT-AI 프로젝트 패키지 구조 (개정안)

적용 결정: ① domain/ 래퍼 유지 — 비즈니스 도메인과 인프라(common/config/security/external/batch)의 시각적 분리. ② external/ 최상위 유지 — 외부 시스템(Kakao OAuth, LLM)을 한곳에 모아 보안 감사·시크릿 관리. ③ audit 직접 의존 허용 — 로거형 횡단 관심사로 client/ 우회 예외 처리.

1. 전체 디렉터리 트리

```
repo-root/
├── flutter-app/
├── qtai-server/
│   ├── .github/
│   │   ├── pull_request_template.md
│   │   └── workflows/
│   │       └── qt-ai-ci.yml
│   ├── apis/
│   │   └── api-v1/
│   │       └── openapi.yaml
│   ├── data/
│   │   └── bible-sources/
│   │       └── README.md
│   ├── workspaces/
│   │   ├── Lead_강태오/
│   │   │   ├── workflows/
│   │   │   └── reports/
│   │   ├── 강상민/
│   │   │   ├── workflows/
│   │   │   └── reports/
│   │   ├── 김지민/
│   │   │   ├── workflows/
│   │   │   └── reports/
│   │   ├── 김태혁/
│   │   │   ├── workflows/
│   │   │   └── reports/
│   │   ├── 이승욱/
│   │   │   ├── workflows/
│   │   │   └── reports/
│   │   └── 이지윤/
│   │       ├── workflows/
│   │       └── reports/
│   ├── build.gradle.kts
│   ├── settings.gradle.kts
│   ├── docker-compose.yml
│   └── Dockerfile
├── src/
│   ├── main/java/com/qtai/
│   │   ├── QtAiApplication.java
│   │   ├── common/
│   │   │   ├── exception/
│   │   │   │   ├── BusinessException.java
│   │   │   │   └── ErrorCode.java
│   │   │   └── dto/
│   │   │       └── ApiResponse.java
│   │   ├── config/
│   │   ├── security/
│   │   ├── batch/
│   │   ├── external/
│   │   │   ├── kakao/
│   │   │   │   ├── KakaoOAuthClient.java
│   │   │   │   └── KakaoOAuthClientImpl.java
│   │   │   └── dto/
│   │   │       └── KakaoOAuthClient.java
│   │   └── # Spring/JPA/Web 전역 설정
│   │       ├── # filter, handler, jwt, oauth (횡단 관심사)
│   │       ├── # ai / notification / retention 스케줄 잡
│   │       └── # 외부 시스템 (한곳에 모아 보안 감사·시크릿 관리)
│   │           └── # interface
```



```

├── StudyResponse.java
├── internal/
│   ├── StudyService.java
│   ├── Study.java
│   └── StudyRepository.java
├── client/
│   ├── bible/
│   │   └── GetBibleVerseUseCaseMock.java
│   └── member/
│       └── GetMemberUseCaseMock.java
├── web/
│   └── StudyController.java
├── note/
│   ├── api/
│   │   ├── CreateNoteUseCase.java
│   │   ├── GetNoteUseCase.java
│   │   ├── UpdateNoteUseCase.java
│   │   └── DeleteNoteUseCase.java
│   │   └── dto/
│   │       ├── NoteCreateRequest.java
│   │       └── NoteResponse.java
│   ├── internal/
│   │   ├── NoteService.java
│   │   ├── Note.java
│   │   └── NoteRepository.java
│   ├── client/
│   │   ├── qt/
│   │   │   └── GetQtUseCaseMock.java
│   │   └── member/
│   │       └── GetMemberUseCaseMock.java
│   └── web/
│       └── NoteController.java
├── sharing/
│   ├── api/
│   │   ├── CreateShareUseCase.java # 스냅샷 생성
│   │   ├── GetSharedSnapshotUseCase.java
│   │   ├── RevokeShareUseCase.java
│   │   └── dto/
│   │       ├── ShareCreateRequest.java
│   │       └── SharedSnapshotResponse.java
│   ├── internal/
│   │   ├── SharingService.java
│   │   ├── Share.java
│   │   ├── ShareSnapshot.java # 원본 변경과 분리된 스냅샷
│   │   └── ShareRepository.java
│   ├── client/
│   │   ├── qt/
│   │   │   └── GetQtUseCaseMock.java
│   │   ├── note/
│   │   │   └── GetNoteUseCaseMock.java
│   │   ├── study/
│   │   │   └── GetStudyUseCaseMock.java
│   │   └── member/
│   │       └── GetMemberUseCaseMock.java
│   └── web/
│       └── SharingController.java
├── report/
│   ├── api/
│   │   ├── CreateReportUseCase.java
│   │   └── GetReportUseCase.java
│   │   └── dto/
│   │       ├── ReportCreateRequest.java
│   │       └── ReportResponse.java
│   ├── internal/
│   │   ├── ReportService.java
│   │   ├── Report.java
│   │   └── ReportRepository.java
│   ├── client/
│   │   ├── sharing/
│   │   │   └── GetSharedSnapshotUseCaseMock.java
│   │   └── member/
│   │       └── GetMemberUseCaseMock.java

```

```

├── web/
│   └── ReportController.java
├── notification/
│   ├── api/
│   │   ├── SendNotificationUseCase.java      # 타 도메인/배치가 호출
│   │   ├── ListNotificationUseCase.java
│   │   ├── MarkAsReadUseCase.java
│   │   └── dto/
│   │       ├── NotificationSendRequest.java
│   │       └── NotificationResponse.java
│   ├── internal/
│   │   ├── NotificationService.java
│   │   ├── Notification.java
│   │   └── NotificationRepository.java
│   ├── client/
│   │   └── member/
│   │       └── GetMemberUseCaseMock.java
│   └── web/
│       └── NotificationController.java
├── praise/
│   ├── api/
│   │   ├── CreatePraiseUseCase.java
│   │   ├── ListPraiseUseCase.java
│   │   └── dto/
│   │       ├── PraiseCreateRequest.java
│   │       └── PraiseResponse.java
│   ├── internal/
│   │   ├── PraiseService.java
│   │   ├── Praise.java
│   │   └── PraiseRepository.java
│   ├── client/
│   │   └── member/
│   │       └── GetMemberUseCaseMock.java
│   └── web/
│       └── PraiseController.java
├── mission/
│   ├── api/
│   │   ├── StartMissionUseCase.java
│   │   ├── CompleteMissionUseCase.java
│   │   ├── ListMissionUseCase.java
│   │   └── dto/
│   │       ├── MissionStartRequest.java
│   │       └── MissionResponse.java
│   ├── internal/
│   │   ├── MissionService.java
│   │   ├── Mission.java
│   │   └── MissionRepository.java
│   ├── client/
│   │   ├── member/
│   │   │   └── GetMemberUseCaseMock.java
│   │   ├── qt/
│   │   │   └── GetQtUseCaseMock.java
│   └── web/
│       └── MissionController.java
├── ai/
│   ├── api/
│   │   ├── GenerateAiResponseUseCase.java    # qt가 호출 (단발성)
│   │   └── dto/
│   │       ├── AiPromptRequest.java
│   │       └── AiResponse.java
│   ├── internal/
│   │   ├── AiService.java                    # external.llm.LlmClient 직접 주입
│   │   └── AiCallLog.java                   # 감사용 (영속 응답 X)
│   ├── client/
│   │   └── qt/
│   │       └── GetQtUseCaseMock.java         # 컨텍스트 조회용
│   └── web/
│       └── AiController.java
├── admin/
│   └── api/

```



2. 호출 흐름

같은 패키지든 다른 패키지든 동일하게 다음 순서로 흐름입니다: Controller (web) → UseCase (api, interface) → Service (internal) → Repository (internal).

타 도메인을 부를 땐 자기 client/〈상대도메인〉에 어댑터(또는 Mock)를 두고 인터페이스에만 의존합니다. 상대 도메인이 아직 구현되지 않았다면 XxxUseCaseMock.java로 임시 구현해두고, 실제 구현이 머지되면 @Primary Bean이 우선되도록 자연스럽게 전환됩니다.

예외 두 가지 — (1) audit은 모든 도메인이 호출하는 횡단 관심사라 각 Service에서 WriteAuditLogUseCase를 직접 주입합니다. (2) external/*(KakaoOAuthClient, LlmClient 등)은 client/ 우회 없이 도메인 Service에서 직접 주입합니다.

3. 도메인별 의존 요약

member	(leaf — 의존 없음, external/kakao 직접 사용)
bible	(leaf — 의존 없음)
qt	→ bible, member, ai
study	→ bible, member
note	→ qt, member
sharing	→ qt, note, study, member (스냅샷 생성 시)
report	→ sharing, member
notification	→ member
praise	→ member
mission	→ member, qt
ai	→ qt (컨텍스트), external/llm 직접 사용
admin	→ member, sharing, report 등 다수 도메인
audit	(모든 도메인이 호출 — 호출 경로 예외)

4. 다음 단계 (이관 순서 권장)

도메인 13개를 한 PR에 모두 옮기면 충돌이 심합니다. 도메인당 1 PR로 분리하고, 의존이 적은 leaf부터 시작하길 권장합니다 — bible → member → audit → qt → study → note → praise → notification → mission → ai → sharing → report → admin. 그 전에 (a) 폴더 이동 PR(.github/apis/data/workspaces → qtai-server/)과 (b) CI 워크플로의 paths 트리거를 qtai-server/ 기준으로 수정하는 PR을 먼저 머지하세요.